

Latent Graph Shaping

A reward-free, calibrated distance from LeWM’s latent,
deployed as a policy-invariant potential for offline goal-conditioned RL

Experiment proposal

28 August 2026

1 What this proposes, in one paragraph

Offline RL’s distinctive capability over behaviour cloning is *stitching*: recombining sub-trajectories $A \rightarrow B$ and $B \rightarrow C$ into $A \rightarrow B \rightarrow C$. Standard algorithms obtain this only indirectly, through many steps of TD bootstrapping. We propose to compute it directly and in one pass: encode dataset states with a frozen pretrained LeWM encoder, build a graph whose edges are (i) real recorded transitions and (ii) cross-trajectory *identifications* between states the latent says are the same, and run Dijkstra. The resulting distance $d_{\text{graph}}(s, g)$ is then supplied to an off-the-shelf offline algorithm as a potential-based shaping term, which *cannot change the optimal policy* (Ng, Harada & Russell 1999) — so a bad graph costs speed, never correctness. Nothing is added to the RL inner loop. No reward labels are used anywhere in the construction.

The identification threshold is not hand-picked: it comes from the same χ^2 one-step-displacement null already used elsewhere in this project, with the correlation $\hat{\rho}$ estimated from consecutive dataset frames.

Status. The gate experiment (B0, Section 3.1) is done and passed decisively on Two-Room with the real pretrained checkpoint. B1, B2, B3, and B4 are also done (Sections 3.2 to 3.5), including a live-rollout confirmation that the B3/B4 value-ranking win converts into real task-success (Section 3.7) and a B4 extension confirming that B1’s required edge-count cap is a net improvement, not a cost, at the RL-training level (Section 3.6). A live-rollout confirmation of that extension (Section 3.8) qualifies the proxy-metric result: k -capping over baseline holds up, but k -capping beating the *full* graph specifically was a proxy-metric artifact. Section 4 then transfers the whole approach to a second environment, Push-T: the gate check (Section 4.1) is done, and passes via the kill criterion’s named fallback rather than the primary calibrated path; the RL-training pipeline and its live-rollout confirmation are also done, across several rounds of diagnosis and fixes summarised in that section.

2 Method

2.1 Construction

Algorithm 1 Latent graph construction

Require: dataset trajectories with frames $\{o_i\}$; frozen pretrained encoder $\text{enc}(\cdot)$; calibration quantile q

Ensure: a weighted graph G over encoded landmarks, queryable as $d_{\text{graph}}(s, g)$

- 1: $\mathbf{z}_i \leftarrow \text{enc}(o_i)$ for every frame $\triangleright d = 192$; one forward pass, cached once; subsample to landmarks for large datasets
 - 2: $\hat{\rho} \leftarrow \frac{1}{d} \mathbb{E}_{\mathcal{D}}[\mathbf{z}_i^\top \mathbf{z}_{i+1}]$ $\triangleright$ one-step displacement correlation, estimated from the data
 - 3: $\varepsilon^2 \leftarrow 2(1 - \hat{\rho}) F_{\chi_d^2}^{-1}(q)$ $\triangleright$ small- q (left) tail of the one-step-displacement null
 - 4: **for** each recorded trajectory **do**
 - 5: **for** each consecutive pair of frames $(i, i+1)$ **do**
 - 6: add directed *transition* edge $(i, i+1)$, weight 1 $\triangleright$ exact and free — it is the data
 - 7: **end for**
 - 8: **end for**
 - 9: **for** each cross-trajectory pair (i, j) **do**
 - 10: **if** $\|\mathbf{z}_i - \mathbf{z}_j\|^2 < \varepsilon^2$ **then**
 - 11: add undirected *identification* edge (i, j) , weight 1 $\triangleright$ weight 0 breaks the method — Section 3.1
 - 12: **end if**
 - 13: **end for** **return** $d_{\text{graph}}(s, g) \leftarrow$ Dijkstra shortest path from s to g on G $\triangleright$ milliseconds per query
-

An identification edge admits pairs strictly closer in latent space than a typical single step ($\varepsilon^2 = 114$ vs. mean one-step $\|\Delta \mathbf{z}\|^2 = 158$ on Two-Room) — weighting them at 1 step is a conservative upper bound on their true cost, and requires no further tuning.

2.2 How it is consumed

Algorithm 2 Consuming d_{graph} in offline RL — two modes tested, in trial order

Require: $d_{\text{graph}}(s, g)$ from Algorithm 1; goal-conditioned value/critic $V(s, g)$, $Q(s, a, g)$; discount γ

- 1: **procedure** MODE3-POTENTIALSHAPING(s, s', g) $\triangleright$ tried first — recommended *a priori*
 - 2: $\Phi(s) \leftarrow -d_{\text{graph}}(s, g)$ **return** $r(s, s') + \gamma \Phi(s') - \Phi(s)$ $\triangleright$ policy-invariant for any Φ (Ng, Harada & Russell 1999); B3: loses on 4/5 tiers
 - 3: **end procedure**
 - 4: **procedure** MODE2-AUXREGRESSION(s, g) $\triangleright$ tried second
 - 5: $\hat{v} \leftarrow -(1 - \gamma^{d_{\text{graph}}(s, g)}) / (1 - \gamma)$ $\triangleright$ pseudo-value under pure geometric discounting **return**
 $\lambda_{\text{aux}} (V(s, g) - \hat{v})^2$ $\triangleright$ added to V 's ordinary loss; TD objective itself untouched; B3: wins at every tier tested
 - 6: **end procedure**
-

Mode 3 was the point of the design going in: potential-based shaping’s policy-invariance guarantee means a bad graph can only cost speed, never correctness, since nothing in the RL objective searches against Φ the way an adversarial optimisation would. That safety property held. What did not hold was the assumption that safety plus a very accurate Φ would translate into a speed-up in practice — mode 2, tried second, is the one B3 finds actually works.

Table 1: Experiment ladder.

	Question	Method	Kill criterion
B0	Does the graph metric beat raw latent distance?	Spearman vs. oracle geodesic, Two-Room.	Done — passed (0.95 vs. 0.44).
B1	Is the graph well-behaved, and does it survive scale?	Landmark-count scaling (50/300/1000 eps); successor-consistency filter vs. false-edge rate; connectivity/fragmentation vs. q ; degree distribution.	Done — mixed . Quality survives scale; edge count does not (Section 3.2).
B2	Is this about <i>LeWM</i> , or about any latent?	Swap the encoder, same pipeline, same q : random-init ViT, raw pixels, PCA-192 on the pooled pixels (done); DINO-WM, PLDM, LeJEPa (blocked, dead Drive link, no alternative source).	Partially done — did not fire . Training matters; whether it’s <i>SIGReg</i> specifically remains open (Section 3.3).
B3	Does it actually help an RL agent?	GCIQL / GCIVL $\pm$ graph-potential shaping. Metrics: steps-to-threshold (the speed claim) and final success rate. Across data-quality tiers (expert / mixed / subsampled).	Scoped pilot done — reward shaping mostly did not fire, but a fix does . See Sections 3.4 and 3.7.
B4	Is the potential doing the work, or just any potential?	Ablate the distance source fed to mode 2 (auxiliary regression, the mechanism B3 found actually works): none (baseline) / raw Euclidean / graph geodesic / true-geodesic oracle. All six tiers, 3 seeds.	Done — kill criterion did not fire . Graph beats Euclidean at every tier, margin growing with data (Section 3.5). Extension: k -capped graphs (k4/k8) beat even the uncapped graph at every tier on the Spearman proxy (Section 3.6); on real rollout success they instead match the full graph and still clearly beat baseline (Section 3.8).

(continued on next page)

(Experiment ladder, continued)

Question	Method	Kill criterion
Push-T Does the whole approach transfer to a second environment?	Repeat B0’s gate check and B3’s data-tier sweep on Push-T, plus a live-rollout confirmation; q held fixed at the Two-Room value initially.	Done — transfers, with a per-environment calibration fix and several diagnosed fixes along the way. See Section 4 for the full account (gate check, RL-training, live-rollout confirmation, and a stitching-specific showcase).

3 Experiments

B0, B1, B2, and B4 are complete; B3 is complete in its scoped form (GCIQL only, a value-ranking proxy metric, plus a later live-rollout confirmation at the real task-success level). The table above summarises all five, all on Two-Room; each subsection below walks through one experiment’s methodology, results, and a brief takeaway, in that order. Section 4 then asks the same questions of a second environment, Push-T, as its own section.

3.1 B0: does the graph metric beat raw latent distance?

Methodology.

Algorithm 3 B0 methodology

- 1: **Checkpoint:** real pretrained `quentin11/lewm-tworoome` $\triangleright$ verified not collapsed: off-diag. cosine ≈ 0.03 vs. ≈ 0.9999 for the local epoch-1 checkpoint
 - 2: **Data:** 300 episodes $\rightarrow$ 27,675 encoded landmarks, $\hat{\rho} = 0.584$
 - 3: **Graph:** build via Algorithm 1, $q = 10^{-3}$
 - 4: **Ground truth:** grid-Dijkstra oracle over recorded (x, y) positions $\triangleright$ wall geometry read empirically off trajectories, not env source defaults
 - 5: **Eval set:** 3,999 held-out query pairs
 - 6: **Metric:** Spearman(d_{graph} , true distance) vs. Spearman($\|\mathbf{z}_i - \mathbf{z}_j\|$, true distance)
 - 7: **Robustness check:** repeat for $q \in \{0.5, 0.05, 0.01, 10^{-3}, 10^{-6}, 10^{-9}\}$, edge weight held fixed
-

Results.

Performance is *not* flat in q — it improves monotonically as q tightens, tracking the false-edge rate. Every value across five decades ($q \leq 10^{-2}$) still gives overall ≥ 0.93 .

Two implementation bugs were caught during this process by measuring things that could have been assumed, and both would have silently produced a wrong negative result: (i) weighting identification edges at 0 (“same state, free hop”) makes the graph *worse* than raw Euclidean distance (cross-room 0.21 vs. 0.66), because Dijkstra chains many free ~ 4 px hops into large illegitimate shortcuts —

Table 2: B0 result. Spearman correlation with true wall-respecting distance, at the principled setting (identification weight = 1, $q = 10^{-3}$). All $p \approx 0$.

	raw latent Euclidean	graph geodesic
overall	0.44	0.95
same-room	0.33	0.93
cross-room (through the door)	0.66	0.94

Table 3: Sensitivity to the calibration quantile q . “False edge” = identification edge whose true geodesic separation exceeds two environment steps.

q	ε^2	avg. degree	overall	cross-room	false edges
0.5	159.2	275	0.839	0.709	6.8%
0.05	134.0	223	0.900	0.829	4.6%
0.01	124.3	205	0.934	0.905	4.5%
10^{-3}	114.1	186	0.946	0.930	3.4%
10^{-6}	93.9	151	0.966	0.981	2.5%
10^{-9}	80.5	128	0.972	0.986	2.2%

charging one step per hop fixes it; (ii) the χ^2 tail direction was initially inverted ($F^{-1}(1 - \alpha)$ instead of $F^{-1}(q)$), which makes the threshold *looser* as the nominal level tightens and inflated average degree from 32 to 27,580 as α shrank — caught by plotting edge count against the knob and noticing the trend ran backwards.

Takeaways.

- Graph geodesic decisively beats raw latent distance (0.95 vs. 0.44 overall Spearman), including on same-room pairs where there is no topological reason for it to help — suggesting it also denoises the latent, not merely routes around the wall.
- The result is robust, not flat: quality tracks q monotonically, but the defensible claim is “robust across five decades” rather than “a single untuned α works.”
- Two edge-construction bugs (zero-weight identifications; inverted χ^2 tail) were caught by testing before they could produce a silent wrong negative — both fixed.

3.2 B1: is the graph well-behaved, and does it survive scale?

Methodology.

Algorithm 4 B1 methodology

- 1: **Base setup:** same checkpoint and pipeline as B0, $q = 10^{-3}$ fixed, identification weight fixed at 1
 - 2: **Landmark scaling:** repeat Algorithm 1 at $N \in \{50, 300, 1000\}$ episodes; record avg. degree, edge count, overall/cross-room Spearman
 - 3: **for** each identification edge (i, j) with $\cos(a_i, a_j) > 0.7$ **do**
 - 4: **successor-consistency test:** also require successors $(i+1, j+1)$ pass the identification test; drop (i, j) if they don't
 - 5: **end for**
 - 6: **Connectivity:** sweep q from 0.5 down to 10^{-9} ; record fragmentation and cross-room reachability
 - 7: **Degree distribution:** at $q = 10^{-3}$, report mean / p99 / max degree
-

Results.

Table 4: Scaling with landmark budget. Quality degrades gently, edge count does not.

episodes	landmarks	avg. degree	overall	cross-room	id. edges
50	4,595	32	0.966	0.984	74,326
300	27,675	186	0.946	0.930	2,578,506
1000	92,552	578	0.930	0.887	26,744,539

Overall Spearman only drifts $0.966 \rightarrow 0.930$ across a $20\times$ landmark range, staying far above raw Euclidean (0.44–0.49) throughout. Average degree, however, scales roughly linearly with landmark count, so total identification edges scale as $n_{\text{id}} \sim n_{\text{landmarks}}^{1.96}$ — essentially quadratic, extrapolating to $\sim 1.5 \times 10^{10}$ edges at Push-T’s 2.3M frames.

Successor-consistency filtering: only **25.6%** of identification edges were even testable under the action-similarity condition, because most identified pairs have near-uncorrelated actions (mean cosine ≈ 0.02) — Two-Room’s target is never rendered into the pixel observation, so two episodes with the agent at the same spot but different (invisible) goals head in unrelated directions. Of the testable 25.6%, 33% failed and were dropped (8.4% of all identification edges). Net effect: false-edge rate $3.7\% \rightarrow 3.8\%$ (unchanged), downstream Spearman $0.9455 \rightarrow 0.9456$ (unchanged).

Connectivity is full at every q tested, 0.5 down to 10^{-9} — cross-room reachability stays at 100% throughout, though this is not really identification edges’ doing: 446/500 sampled episodes already cross the door within their own single recorded trajectory. Degree distribution at $q = 10^{-3}$ is clean: mean 186, p99 423, max 657 — a smooth tail, zero pathological hubs.

Takeaways.

- Quality does not collapse with scale — the originally-written B1 kill criterion did not fire.
- But edge count explodes ($\sim n^{1.96}$), extrapolating to infeasibility at Push-T scale. **This is the real kill criterion, and it already fired:** a top- k -per-node cap is now a prerequisite for the Push-T transfer work (Section 4), not an optional efficiency tweak.
- Successor-consistency filtering does not help on Two-Room: only 25.6% of edges are even testable (the invisible goal starves action-similarity of signal), and the net effect on false-edge rate and

Spearman is negligible. May work better where the observation determines the policy more fully — should not be assumed for Push-T without checking.

- Identification edges are doing metric work (B0’s Spearman gain) on Two-Room, not connectivity work — transition edges alone already carry most of the room-crossing connectivity here.

3.3 B2: is this about *LeWM*, or about any latent?

Methodology.

Algorithm 5 B2 methodology

- 1: **Fixed:** same 300 episodes, same frames, same pipeline ($q = 10^{-3}$, identification weight 1)
 - 2: **for** $\text{enc} \in \{\text{LeWM (pretrained)}, \text{random-init ViT-tiny, raw pixels (16}\times\text{16 pooled), PCA-192 on the pooled pixels}\}$ **do**
 - 3: re-run Algorithm 1 and B0’s evaluation with this encoder in place of LeWM
 - 4: **end for**
 - 5: **PCA basis:** 192 components, fit via IncrementalPCA over the *entire* dataset (920,809 frames, all 10,000 episodes) — not just the 300-episode landmark subset it is then evaluated on
 - 6: **if** identification-edge count $> 10^7$ at 300 episodes **then**
 - 7: fall back to the same four encoders on a 10-episode subset (same episodes, same reused encodings, no re-encoding) ▷ avg. degree scales $\sim$ linearly with landmark count here, unlike LeWM (B1), so a smaller subset is tractable
 - 8: **end if**
 - 9: **Scope note:** random-init, raw pixels, and PCA test the *weak* form (does training/dimensionality reduction alone matter), not the *sharp* form (is it SIGReg specifically) ▷ DINO-WM/PLDM/LeJEPa baselines, which would test the sharp form, blocked by a dead Drive link, confirmed dead with no alternative source
-

Results.

Table 5: B2 encoder swap. Squared-distance scale and graph Spearman vary enormously by encoder.

encoder	d	$\hat{\rho}$ (adjacent)	overall (graph)	same-room	cross-room
LeWM (pretrained)	192	0.584	0.946	0.932	0.930
random-init ViT-tiny	192	1.000	−0.015	0.087	−0.192
raw pixels (16×16 pooled)	768	0.810	intractable (see below)		
PCA-192 (on the pooled pixels)	192	0.005	intractable (see below)		

Random-init ViT: $\hat{\rho} \approx 1.000$ means adjacent frames map to nearly identical vectors regardless of content — a known failure mode of deep random (untrained) networks. Both raw Euclidean and graph-geodesic distance score at or below zero correlation with true distance.

Raw pixels: 381,644,170 identification edges at $q = 10^{-3}$ — more than LeWM (2.6M) and random-ViT (101M) combined — would need ~ 6 GB just to materialize the edge-index arrays, which crashes (a hard native segfault, not a catchable Python exception) on this machine. Sweeping q down to 10^{-80} changed *nothing*: identical edge count every time. Random cross-episode pairs in 16×16-pooled pixel space have a median squared distance of 2.24 — the fixed wall/door background

dominates the 768-dimensional vector so completely that “adjacent” and “unrelated” pairs are barely distinguishable in Euclidean terms at any threshold.

PCA-192: motivated by exactly that diagnosis — PCA’s entire job is finding the directions that actually vary and discarding near-constant ones (the fixed background), so it looked like a near-free candidate fix. It is not: **381,644,170** identification edges — indistinguishable from raw pixels’ count to six significant figures — and a median cross-episode squared distance of **2.2376**, essentially identical to raw pixels’ 2.2380. $\hat{\rho}$ drops to 0.005 (versus raw pixels’ 0.810), meaning adjacent frames are no longer even weakly correlated once projected. In hindsight this is the expected result, not a surprising one: PCA is a *linear*, near-lossless map (192 of 768 components already capture 99.97% of variance) — it re-expresses the same raw-pixel geometry in a rotated, whitened basis, but introduces no new invariance, so a separation problem that is already present in that geometry survives the projection essentially unchanged.

Edge count for raw pixels/PCA scales very differently from LeWM’s with landmark count: B1 found LeWM’s average degree stays roughly *constant* as data grows (32 at 50 episodes, 186 at 300, 578 at 1000), so total edges scale sub-quadratically; raw pixels and PCA instead have avg. degree scaling roughly *linearly* with landmark count (checked directly: 893 at 10 episodes, 9,289 at 100, 18,544 at 200), so total edges scale roughly quadratically and 10 episodes is already the largest tractable subset on this machine. Re-running all four encoders on that same 10-episode subset (same episodes throughout, reusing the already-computed encodings) gives a real, apples-to-apples number instead of just “intractable”:

Table 6: Small-scale (10 episodes, 992 frames) comparison, same episode subset for all four encoders.

encoder	d	overall (eucl.)	overall (graph)	same-room (graph)	cross-room (graph)
LeWM (pretrained)	192	0.438	0.938	0.944	0.932
random-init ViT-tiny	192	−0.108	−0.108	0.064	−0.424
raw pixels	768	0.619	−0.118	−0.173	−0.001
PCA-192	192	0.618	−0.118	−0.173	−0.001

This is sharper than “intractable,” not just a fallback: at a scale where the graph geodesic *can* be computed, raw pixels and PCA’s graph distance is not merely uncorrelated with truth, it is *worse than doing nothing* — raw Euclidean distance in the same space scores 0.619 / 0.618, but routing through the graph drags it down to −0.118 for both, again matching to three decimal places. With avg. degree 893 at only 992 nodes, the identification edges are dense enough that Dijkstra chains illegitimate shortcuts almost everywhere — the same mechanism already diagnosed in B0’s zero-weight-edge bug (Section 3.1), except here it is not a weighting mistake, just too many genuinely-under-threshold-but-not-actually-nearby pairs for the threshold to mean anything.

Takeaways.

- Random-init ViT is collapsed, not merely worse ($\hat{\rho} \approx 1$, Spearman ≤ 0) — evidence *for* the trained representation doing real work, not just architecture. The sharp kill criterion did not fire.
- Raw pixels and PCA are intractable at 300 episodes but, checked at the largest tractable subset (10 episodes), the graph is not just unhelpful — it is actively *worse than raw Euclidean distance* in the same space, for both, to three decimal places.

- PCA does not fix raw pixels’ failure: it reproduces its edge count, squared-distance distribution, and small-scale graph Spearman almost exactly, because a linear, variance-preserving projection cannot introduce the invariances (to the fixed background, in particular) that the problem actually needs — evidence that the failure is about representation *content*, not merely dimensionality.
- Scope caveat: none of random-init, raw pixels, or PCA test whether it’s *SIGReg* specifically versus another trained objective — that comparison remains blocked (dead Drive link, confirmed with no alternative source) and open.

3.4 B3: does it actually help an RL agent?

Methodology.

Algorithm 6 B3 methodology: setup and the initial shaping test

- 1: **Algorithm:** GCIQL only (not GCIVL); expectile loss reused verbatim from `stable_worldmodel.wm.gcrl.ExpectileLoss`
 - 2: **Split:** 80/20 train/test episodes per tier $\triangleright$ graph built over *all* episodes, since Φ is a fixed deterministic function of the frozen encoder, not fit to minimise eval error
 - 3: **HER:** tuples drawn only from train episodes
 - 4: **Eval sets:** 1,000 fixed train pairs, 1,000 fixed held-out test pairs; true distances precomputed once
 - 5: **Tiers:** `expert_{10,25,50,100}` (real episodes only); mixed = 50 real + 50 (later 220) live noisy rollouts, `ExpertPolicy(action_noise=0.4, action_repeat_prob=0.15)`
 - 6: **Conditions:** baseline (−1/0 TD, untouched) vs. baseline + MODE3-POTENTIALSHAPING (Algorithm 2)
 - 7: **Training:** 50,000 steps, evaluate every 50 steps; checkpoint every eval step, resume on interrupt
 - 8: **Metric:** peak held-out-test Spearman, not final-step value $\triangleright$ every condition on every tier peaks early then degrades from a shared TD/overfitting instability
-

Algorithm 7 B3 methodology: mechanism diagnosis and fix attempts, once shaping was found to fail

- 1: **Graph-pollution check:** compare $\text{Spearman}(\Phi, \text{true})$ on expert-only pairs, with vs. without noisy-tier nodes in the graph
 - 2: **Disconnection check:** connectivity, reachability, and identification-edge degree, split by node origin (expert vs. noisy)
 - 3: **Reward-noise-mismatch check:** mean/std of $\gamma\Phi(s') - \Phi(s)$ on HER tuples, split by origin episode
 - 4: **Peak-snapshot inspection:** re-train keeping peak-step (not final-step) weights; inspect $V(s, g)$ shape against true-distance bin
 - 5: **Fix attempt 1:** $\lambda \cdot (\gamma\Phi(s') - \Phi(s))$ for $\lambda \in \{0.1, 0.15, 0.3, 0.5\}$ $\triangleright$ preserves the exact potential-based structure
 - 6: **Fix attempt 2:** replace shaping with MODE2-AUXREGRESSION (Algorithm 2), $\lambda_{\text{aux}} = 0.3$, all six tiers, 3 seeds
-

Results.

Table 7: B3 across data tiers: peak held-out-test Spearman, mean $\pm$ std over 3 seeds.

tier	baseline	shaped	shaped – baseline
expert (10 eps)	0.303 ± 0.013	0.221 ± 0.018	-0.082
expert (25 eps)	0.197 ± 0.008	0.145 ± 0.029	-0.052
expert (50 eps)	0.272 ± 0.030	0.212 ± 0.049	-0.061
expert (100 eps)	0.306 ± 0.024	0.328 ± 0.028	$+0.022$
mixed (50+50 eps)	0.340 ± 0.013	0.205 ± 0.012	-0.135

Shaping (mode 3) wins at exactly one of five tiers: the largest, cleanest one (100 clean expert episodes, shaped peak higher in under half the steps). Everywhere else it loses, worst on the mixed-quality tier. Scaling the mixed tier up further (**mixed_large**: same 50 real episodes, 220 noisy episodes instead of 50, $\sim 10,400$ landmarks) makes this worse, not better: baseline nearly doubles ($0.340 \rightarrow 0.575$), shaping barely moves ($0.205 \rightarrow 0.266$) — the gap widens from -0.135 to -0.309 , the worst of any tier tested.

Mechanism checks: Spearman(Φ , true distance) on identical expert-only pairs is the same (0.969 vs. 0.967) whether or not the graph also contains noisy-tier nodes — not graph pollution. One connected component, 100% of noisy-origin nodes reachable, comparable identification-edge degree by origin (29.5 vs. 33.4) — not disconnection. The shaping increment $\gamma\Phi(s') - \Phi(s)$ has mean 0.46, std 0.68 on real-expert HER tuples vs. mean 0.75, std 0.47 on noisy-collector tuples: real trajectories track monotonic graph-distance progress toward HER-sampled goals *less* consistently (91 avg. steps/episode vs. ~ 25 for the direct waypoint-following collector) even though Φ itself stays uniformly accurate — a population-conditional reward-noise mismatch that mode 3 folds into the reward, where TD bootstraps and compounds it; plain TD’s $-1/0$ reward has no such channel. Peak-snapshot inspection: baseline’s peak $V(s, g)$ is nearly flat (range 0.08) but ranks decently ($\rho = 0.21$); shaped’s peak V has $\sim 10\times$ more range (0.81) but is non-monotonic — correct near and far, reversed at mid-range distances — and that reversal is exactly where it loses ($\rho = 0.17$).

Fix attempt 1 (λ -scaled mode 3): on the mixed tier, $\lambda = 0.1 \rightarrow 0.328$, $0.15 \rightarrow 0.323$, $0.3 \rightarrow 0.311$, $0.5 \rightarrow 0.293$ — recovers $\sim 90\%$ of the $\lambda = 1$ deficit (0.205) but never crosses baseline’s 0.340.

Table 8: B3 final: peak held-out-test Spearman, mean $\pm$ std over 3 seeds, all six tiers.

tier	baseline	shaped (mode 3)	aux. reg. (mode 2)	Δ (mode 2)
expert_10	0.303 ± 0.013	0.221 ± 0.018	0.316 ± 0.039	$+0.013$
expert_25	0.197 ± 0.008	0.145 ± 0.029	0.351 ± 0.028	$+0.154$
expert_50	0.272 ± 0.030	0.212 ± 0.049	0.383 ± 0.012	$+0.111$
expert_100	0.306 ± 0.024	0.328 ± 0.028	0.646 ± 0.010	$+0.339$
mixed	0.340 ± 0.013	0.205 ± 0.012	0.490 ± 0.016	$+0.150$
mixed_large	0.575 ± 0.007	0.266 ± 0.026	0.908 ± 0.002	$+0.334$

Fix attempt 2 (mode 2, auxiliary regression) beats baseline at every tier tested, by a margin that grows with data — the precise inverse of mode 3’s pattern. Only expert_10 is a wash (seed ranges overlap); from expert_25 up there is no overlap in either direction. On **mixed_large**, mode 2 reaches 0.908, beating baseline’s already-strong 0.575 by $+0.334$.

Takeaways.

- Potential-based shaping (mode 3) wins only at the largest/cleanest tier, loses everywhere else, and gets *worse* rather than better as more heterogeneous data is added — inverting the proposal’s own hypothesis that shaping would help most when data is scarce.
- Root cause: not graph noise (Φ stays ≥ 0.96 accurate throughout) but a population-conditional reward-noise mismatch that TD bootstrapping compounds across a trajectory; plain $-1/0$ TD has no such channel, which is why it alone benefits from added heterogeneous data.
- Scaling mode 3’s magnitude down recovers most, not all, of the deficit — confirms a magnitude/variance issue, not a fix worth shipping.
- The fix that works: stop routing Φ through the reward at all. Mode 2 (direct, non-bootstrapped auxiliary regression toward a Φ -derived pseudo-value) beats baseline at every tier, margin growing with data (+0.34 at expert_100 and mixed_large).
- Label efficiency (fewer reward labels than baseline, same d_{graph}) is a natural extra axis for B3 that costs nothing extra to measure — not yet run.

3.5 B4: is the potential doing the work, or just any potential?

Methodology.

Algorithm 8 B4 methodology

- 1: **Fixed:** MODE2-AUXREGRESSION loop (Algorithm 2), target form $-(1-\gamma^{d(s,g)})/(1-\gamma)$, $\lambda_{\text{aux}} = 0.3$
 - 2: **for** $d(s, g) \in \{\text{euclidean: } \|\mathbf{z}_s - \mathbf{z}_g\|, \text{graph: } d_{\text{graph}}, \text{oracle: true wall-respecting pixel distance}\}$ **do**
 - 3: train and evaluate as in MODE2-AUXREGRESSION, all six tiers, 3 seeds each
 - 4: **end for**
 - 5: **Note:** euclidean is B0’s known-bad proxy; oracle is not deployable (needs privileged position info) — a noise-free ceiling
-

Results.

Table 9: B4: peak held-out-test Spearman by distance source fed to mode 2, mean over 3 seeds.

tier	baseline	euclidean	graph	oracle (ceiling)
expert_10	0.303	0.314	0.316	0.452
expert_25	0.197	0.267	0.351	0.435
expert_50	0.272	0.273	0.383	0.562
expert_100	0.306	0.317	0.646	0.728
mixed	0.340	0.381	0.490	0.641
mixed_large	0.575	0.443	0.908	0.931

Graph beats euclidean at every tier, gap growing with data (mixed_large: graph 0.908, more than double euclidean’s 0.443). Euclidean-mode-2 loses to baseline at mixed_large (0.443 vs. 0.575). Graph closes in on the oracle ceiling as data scales: the gap is largest at expert_50 (0.383 vs. 0.562, a gap of 0.18) and shrinks to 0.023 at mixed_large (0.908 vs. 0.931).

Takeaways.

- Graph clearly beats Euclidean everywhere, and the gap grows with data — the core thesis holds under the mechanism that actually works.
- The auxiliary-regression mechanism is not sufficient by itself: regressing toward a bad distance proxy (euclidean) can lose to doing nothing at all. It works specifically because the graph gives it something accurate to regress toward.
- Graph calibration quality itself improves with more data, converging toward the ground-truth ceiling rather than plateauing below it.
- Kill criterion (“raw Euclidean shaping matches graph shaping”) did not fire.

3.6 B4 extension: transitions-only and edge-capped graphs

Methodology.

Algorithm 9 B4-extension methodology

- 1: **Motivation:** a local, static (no RL) diagnostic campaign found that (i) same-episode ranking is measurably worse without identification edges, growing with data heterogeneity, and (ii) capping identification edges to each node’s top- k FAISS-HNSW neighbours costs nothing in Spearman quality (often slightly better) even at $k=4$ — the fix B1 flagged as required before attempting Push-T (Section 4). This extension checks both findings at the real RL-training level, not just the static metric.
 - 2: **Fixed:** MODE2-AUXREGRESSION loop (Algorithm 2), identical target form and $\lambda_{\text{aux}} = 0.3$, unchanged from B4
 - 3: **for** $d(s, g) \in \{\text{transonly: } d_{\text{graph}} \text{ with identification edges dropped, k4: } d_{\text{graph}} \text{ capped to top-4 neighbours, k8: capped to top-8}\}$ **do**
 - 4: train and evaluate as in MODE2-AUXREGRESSION, 4 tiers (expert_50, expert_100, mixed, mixed_large), 5 seeds each
 - 5: **end for**
 - 6: **Compute:** IIIT-H Ada cluster, direct-SSH orchestration (no SLURM), 60 runs total
-

Results.

Table 10: B4-extension: peak held-out-test Spearman, mean $\pm$ std over 5 seeds (baseline and full graph repeated from Section 3.5 for reference, 3 seeds each).

tier	baseline	graph (full)	transonly	k4	k8
expert_50	0.272	0.383	0.417 ± 0.034	0.500 ± 0.040	0.463 ± 0.037
expert_100	0.306	0.646	0.646 ± 0.014	0.692 ± 0.016	0.679 ± 0.022
mixed	0.340	0.490	0.446 ± 0.015	0.584 ± 0.023	0.575 ± 0.023
mixed_large	0.575	0.908	0.790 ± 0.015	0.913 ± 0.009	0.915 ± 0.009

k4 and k8 do not merely match the full (uncapped) graph — they beat it, at every tier tested: +0.117 / +0.080 at expert_50, +0.046 / +0.033 at expert_100, +0.094 / +0.085 at mixed, +0.005 / +0.007 at mixed_large (k4/k8 respectively, vs. the full graph). This directly extends the local static finding (capping costs nothing, often helps) to the metric that actually matters.

Transitions-only is competitive at the two pure-expert tiers but falls behind on the **mixed-quality ones**: statistically indistinguishable from the full graph at expert_50 (+0.034) and expert_100 (+0.000), but clearly behind at mixed (−0.044) and mixed_large (−0.118), the gap growing with data heterogeneity. This matches the local static mechanism-probe’s own pattern exactly (same-episode Spearman gap without identification edges was smallest on pure-expert tiers, +0.05, and largest on mixed/mixed_large, +0.12/+0.14) — the two independent diagnostics, one static and one from real training, agree on *where* identification-edge stitching matters most.

Takeaways.

- Edge-count capping is not a quality trade-off to accept for Push-T-scale feasibility — it is a net improvement, confirmed now at the RL-training level, not just the static Spearman diagnostic. This resolves B1’s “top- k cap required before Push-T” prerequisite with a favourable answer rather than an open cost.
- Cross-episode stitching’s value is concentrated on heterogeneous data: on clean expert-only tiers, a transitions-only graph is nearly as good as the full graph; on mixed-quality tiers, the full graph’s identification edges are clearly load-bearing, and the size of that gap tracks the same pattern found statically outside any RL loop.
- Practical implication for Push-T (Section 4): a k -capped graph is not just feasible at that scale, it is now the recommended default even where the uncapped graph would be affordable, since it wins on Two-Room’s own comparison.

3.7 Live-rollout success rate: confirming the B3/B4 result

Methodology.

Algorithm 10 Live-rollout methodology

- 1: **Actor loss (AWR):** $w \leftarrow \text{clip}(\exp(\beta(Q(s, a, g) - V(s, g))), 100), \beta = 3.0; \ell_{\text{actor}} \leftarrow w \cdot \|\pi(s, g) - a\|^2$; advantage computed under `no_grad`
 - 2: **peak_rho:** cheap Spearman-based peak, tracked every 50 steps
 - 3: **peak_success:** live-rollout success rate over a fixed 12-episode subset, tracked every 2,500 steps, using `TwoRoomEnv`’s own criterion (distance-to-goal < 16)
 - 4: **Sweep:** 6 tiers $\times$ {baseline, mode 2} $\times$ 5 seeds = 60 runs, 20,000 steps each $\triangleright$ truncated from 50,000 — all pilot peaks landed by step 12,650
 - 5: **Final eval:** 100 episodes, at both the rho-selected and the success-selected checkpoint
 - 6: **Compute:** IIIT-H Ada cluster
-

Results.

Auxiliary regression beats baseline at 6 of 6 tiers under both selection criteria (up from 5 of 6 on the proxy metric — expert_10 was a genuine wash there, a small consistent win here). Margin grows with data exactly as the Spearman result predicted: negligible at expert_10, +0.09 to +0.19 by expert_100 and mixed_large. Selecting by the success peak gives a small edge over selecting by the rho peak in 10 of 12 rows; **peak_success** is often first observed at step 2,500 while **peak_rho** can be as early as step 300.

Takeaways.

Table 11: Live-rollout success rate, mean over 5 seeds, by checkpoint-selection criterion.

tier	baseline (rho-sel)	baseline (success-sel)	auxphi (rho-sel)	auxphi (success-sel)
expert_10	0.414	0.426	0.406	0.420
expert_25	0.480	0.458	0.490	0.514
expert_50	0.464	0.406	0.476	0.508
expert_100	0.588	0.620	0.712	0.714
mixed	0.402	0.422	0.448	0.472
mixed_large	0.668	0.718	0.788	0.808

- The proxy-metric result converts into real task success: this is direct evidence that the value-ranking win measured throughout B3–B4 was tracking something that mattered for the policy actually extracted from it, not an artifact of the ranking metric.
- The two peaks (rho-based, success-based) genuinely track different things during training and land at different steps — justifying tracking both rather than assuming they coincide.

3.8 Live-rollout success rate: confirming the B4 extension

Methodology.

Algorithm 11 B4-extension live-rollout methodology

- 1: **Fixed:** identical actor (AWR) and dual peak-tracking setup as Section 3.7 — only the distance source fed to mode 2 changes
 - 2: **for** $d(s, g) \in \{\text{transonly}, \text{k4}, \text{k8}\}$ (as defined in Section 3.6) **do**
 - 3: train actor, live-rollout eval as in Section 3.7, 4 tiers (expert_50, expert_100, mixed, mixed_large), 5 seeds each
 - 4: **end for**
 - 5: **Compute:** IIIT-H Ada cluster, direct-SSH orchestration, 60 runs (train $\rightarrow$ rho-eval $\rightarrow$ success-eval chained per run)
-

Results.

Table 12: Live-rollout success rate at the success-selected checkpoint, mean $\pm$ std over 5 seeds (baseline and full graph repeated from Section 3.7 for reference).

tier	baseline	graph (full)	transonly	k4	k8
expert_50	0.406	0.508	0.410 ± 0.025	0.448 ± 0.061	0.492 ± 0.033
expert_100	0.620	0.714	0.538 ± 0.038	0.694 ± 0.023	0.692 ± 0.015
mixed	0.422	0.472	0.380 ± 0.044	0.472 ± 0.065	0.460 ± 0.046
mixed_large	0.718	0.808	0.686 ± 0.038	0.804 ± 0.020	0.792 ± 0.046

Unlike the Spearman-proxy result (Section 3.6), where k4 and k8 beat the full uncapped graph outright at every tier, on real rollout success they instead *match* it closely: k4 ties the full graph exactly at mixed (0.472 vs. 0.472) and comes within 0.02 of it at expert_100 and mixed_large, only clearly trailing at expert_50 (0.448 vs. 0.508). Both k4 and k8 still beat baseline decisively at all four tiers (+0.04 to +0.09). Transitions-only tells the more consequential story: the proxy metric

called it “competitive at the two pure-expert tiers,” but on real rollout success it is at or below baseline at all four (a wash at expert_50, -0.04 to -0.08 elsewhere) — it never wins. Selecting the checkpoint by the rho-peak instead of the success-peak gives the same qualitative picture (k4/k8 close to the full graph and clearly ahead of baseline; tranonly at or below baseline everywhere), so this is not an artifact of the checkpoint-selection criterion.

Takeaways.

- The recommendation to use a k -capped graph as the default construction method (Section 3.6) survives contact with real task performance: k4/k8 remain a clear, consistent win over baseline, even though their edge over the *full* graph specifically was a proxy-metric artifact that does not reproduce at the real-task level.
- Transitions-only’s failure mode is worse in practice than the proxy metric suggested: it does not merely fall behind the full graph on heterogeneous data, it fails to beat doing nothing (baseline) at any tier tested. Identification edges are not optional at the real-task level, even on the pure-expert tiers where the static and Spearman-proxy diagnostics suggested near-parity.
- General lesson for this project: the Spearman-vs-oracle proxy metric is directional but not quantitatively reliable close to the top of the ranking — differences among the strongest few conditions (full graph vs. k4 vs. k8) do not survive to real task success in the same order or magnitude, even though the much larger baseline/tranonly-vs-graph gaps do.

4 Push-T: transferring the approach to a second environment

Everything above is Two-Room. This section repeats the same questions — does the calibrated graph beat raw latent distance, does it help offline RL, does it hold up under a live-rollout success check — on Push-T, a contact-manipulation task with a very different observation and dynamics structure. It is reported as its own arc rather than folded into the Two-Room ladder above, because it did not transfer cleanly on the first attempt: the calibration formula fails outright, the live-rollout protocol initially used was too hard to show any signal at all, and diagnosing the resulting apparent underperformance took several rounds of investigation. Each subsection below is a self-contained result with its own methodology and configuration; later subsections build on fixes established in earlier ones, noted explicitly where relevant.

4.1 Does the identification-edge calibration transfer?

Methodology.

Algorithm 12 Push-T gate-check methodology — Two-Room’s B0 gate, ported to Push-T

- 1: **Checkpoint:** real pretrained `quentin11/lewm-pusht` (downloaded fresh, not the local from-scratch epoch-1 checkpoint, which is confirmed collapsed — Section 3.3-style off-diagonal cosine and participation-ratio check run first)
 - 2: **Landmarks:** 300 episodes (36,824 frames), encoded with the same frozen-encoder pipeline as Algorithm 1
 - 3: **Oracle:** no walls to route around, so ground truth is Euclidean distance in z-scored privileged state $[\text{agent}_{x,y}, \text{block}_{x,y}, \cos \theta_{\text{block}}, \sin \theta_{\text{block}}]$, not a navigation-grid distance
 - 4: **Calibration:** hold $q = 10^{-3}$ fixed at the Two-Room value, as proposed
 - 5: **if** calibrated $\varepsilon^2 \leq 0$ (ρ_{hat} statistically ≥ 1) **then**
 - 6: **fall back** to the named contingency: heuristic (uncalibrated) top- k identification edges, $k = 4$ carried over from Section 3.6 unchanged
 - 7: **end if**
 - 8: compare Spearman(oracle, graph-geodesic) vs. Spearman(oracle, raw latent Euclidean)
-

Results.

The calibrated threshold failed outright, not marginally: $\hat{\rho}$ came out statistically ≥ 1 (measured 1.0009 at 300 episodes, up from an already-pathological 0.9985 at 20 — more landmarks made it *worse*, ruling out a landmark-coverage explanation), forcing $\varepsilon^2 < 0$ and exactly zero identification edges at every quantile tested, at both landmark scales. This is not a checkpoint-collapse artifact: a properly-randomized, large-sample participation-ratio check (the naive small-batch check is biased by temporally correlated sampling and understates it) puts this checkpoint’s effective rank at 84 of 192 dims, at least as healthy as Two-Room’s 37 of 192. The real cause is narrower: adjacent-frame latent displacement is tiny relative to the cross-episode spread on Push-T (squared-distance ratio $\approx 176\times$) versus Two-Room’s $\approx 2.4\times$ — Two-Room’s calibration formula implicitly assumes a margin this checkpoint’s geometry does not have, independent of its overall quality.

Table 13: Push-T: Spearman vs. the privileged-state oracle, 300-episode landmark set, heuristic top-4 identification edges (calibrated threshold failed — see above).

identification-edge weight	raw euclidean	graph geodesic
0.0	0.219	0.319
0.5	0.219	0.518
1.0	0.219	0.537
2.0	0.219	0.540
4.0	0.219	0.534

With the heuristic fallback, the graph beats raw Euclidean by more than $2\times$ (0.540 vs. 0.219 at the best identification-edge weight) — a clear pass of the same gate B0 applied to Two-Room, on 98.3% of sampled pairs (up from under 2% finite under the failed calibrated graph, whose near-total disconnection made most cross-episode comparisons undefined).

Takeaways.

- The kill criterion (“needs per-environment retuning”) fired in its precise, named form — the calibration quantile did not transfer as-is — but the documented fallback (a heuristic, uncalibrated top- k graph) recovers a decisive win over raw Euclidean anyway. This is the weaker-but-still-useful deployment story the proposal anticipated, not a failure of the core idea.

- The failure mode is specific and diagnosable, not a vague “it didn’t work”: it is a mismatch between how large adjacent-frame latent movement is relative to the overall latent spread on the two environments, not a difference in checkpoint quality (which is comparable or better on Push-T by the properly-measured participation ratio).
- $k = 4$ was carried over unchanged from Two-Room’s B4-extension winner, not retuned for Push-T — a natural next check, not yet done.
- This result used only the local machine’s GPU (300-episode encode + graph + eval, a few minutes total) — no Ada allocation needed for B0. The RL-training pipeline result below did need Ada.

4.2 Does the graph help offline RL training?

Methodology.

Algorithm 13 Push-T RL-training methodology — Two-Room’s B3 data-tier sweep, ported to Push-T

- 1: **Fixed:** MODE2-AUXREGRESSION loop (Algorithm 2), identical target form and $\lambda_{\text{aux}} = 0.3$, unchanged from Two-Room’s B3/B4 — `run_condition_resumable` reused with zero code changes, only a Push-T-specific `setup` dict
 - 2: **Graph:** heuristic top-4 identification edges (Section 4.1), built fresh per tier
 - 3: **Tiers:** `expert_10/25/50/100` (real expert episodes only) and `mixed` (50 real expert + 50 WeakPolicy-rollout episodes — `stable_worldmodel`’s built-in weak/unfocused Push-T collection policy, validated to fail the task: 0/20 rollouts reached the goal within 250 steps at every tested setting, confirming it is genuinely low-quality data, not disguised expert data)
 - 4: train and evaluate baseline ($\lambda_{\text{aux}} = 0$) and `auxphi` ($\lambda_{\text{aux}} = 0.3$), all 5 tiers, 3 seeds each, 50,000 steps
 - 5: **Compute:** IIIT-H Ada cluster, direct-SSH orchestration, 30 runs total
-

Results.

Table 14: Push-T: peak held-out-test Spearman, mean over 3 seeds.

tier	baseline	auxphi	Δ
<code>expert_10</code>	0.148	0.178	+0.030
<code>expert_25</code>	0.284	0.330	+0.046
<code>expert_50</code>	0.128	0.182	+0.054
<code>expert_100</code>	0.074	0.144	+0.070
<code>mixed</code>	0.391	0.418	+0.027

Auxphi beats baseline at all 5 of 5 tiers, matching Two-Room’s B3/B4 finding qualitatively. Two things do *not* match Two-Room, and are reported rather than smoothed over: (1) absolute Spearman is far lower here (peaking at 0.42 vs. Two-Room’s 0.91 at its best tier) and (2) it is *non-monotonic* in tier size — `expert_25` (0.284/0.330) outscores both `expert_50` and `expert_100`, the opposite of Two-Room’s clean monotonic improvement with data. This is not a graph-construction artifact: identification-edge count and average degree both scale up sensibly with landmark count (degree $0.05 \rightarrow 0.07 \rightarrow 0.34 \rightarrow 0.50 \rightarrow 0.44$ across `expert_10/25/50/100/mixed`), the same qualitative pattern B1 found healthy on Two-Room. The more likely cause is a fixed training budget (50,000

steps, unchanged from Two-Room) not scaling with the HER-tuple pool size, which grows faster on Push-T than on Two-Room at the same nominal episode count (Push-T’s episodes average 125 steps vs. Two-Room’s 92) — larger Push-T tiers may simply be undertrained relative to smaller ones at a fixed step count, and/or per-tier oracle z-scoring (recomputed fresh per tier, unlike Two-Room’s fixed wall-geometry oracle) makes absolute values not strictly comparable across tiers. Neither is confirmed; both are candidates for a longer training-budget or fixed-oracle-normalization follow-up rather than settled fact.

Takeaways.

- The core claim replicates on a second environment: the graph-based auxiliary-regression mechanism beats baseline everywhere tested, using the same code, the same mechanism, and (for identification edges) the same fallback construction method established in Section 4.1.
- Absolute magnitude and its scaling with data size do *not* replicate cleanly — this is Push-T’s first RL-training result, not yet debugged to the same depth as Two-Room’s multi-round B3/B4 process was, and the non-monotonic tier-size pattern is flagged as open rather than explained away.
- Live-rollout confirmation (the real task-success check that mattered for Two-Room’s headline result, Section 3.7) has not yet been built for Push-T — this proxy-metric result alone should be read with the same caution the B4-extension proxy result eventually needed (Section 3.8).

4.3 Diagnosing why the calibrated threshold fails

Root cause. The calibration formula $\hat{\rho} = \frac{1}{d}\mathbb{E}[\mathbf{z}_t^\top \mathbf{z}_{t+1}]$ (mean inner product between temporally-adjacent same-episode frames, normalized by the raw embedding dimension $d=192$) implicitly assumes two things: that $\mathbb{E}[\|\mathbf{z}\|^2] = d$, and that the embedding’s effective degrees of freedom equal d . Neither holds tightly enough on Push-T. Measured directly: $\mathbb{E}[\|\mathbf{z}\|^2] = 193.2$ (a 0.6% deviation from the assumed 192), and the covariance’s participation ratio — effective dimensionality, not the raw ambient one — is 84 of 192 dims. On Two-Room ($\hat{\rho}=0.584$) this 0.6% normalization error is negligible relative to $1 - \hat{\rho} \approx 0.42$; on Push-T ($1 - \hat{\rho} \approx 0.001$) the same error is large enough to flip the sign, since $\varepsilon^2 = 2(1 - \hat{\rho})F_{\chi_d^2}^{-1}(q)$ collapses to a negative “variance” the moment $\hat{\rho} \geq 1$. This is not a sign the checkpoint is worse on Push-T: an independently-measured, large-sample participation ratio puts it at 84 of 192 effective dims, comparable to Two-Room’s own. The formula’s assumptions break, not the representation.

Is a different distance metric the fix? Tested directly rather than assumed: dot product, cosine similarity (raw and mean-centered), per-dimension standardization, and a PCA-whitened basis, each scored as edge precision (fraction of $k=4$ cross-episode nearest neighbors whose true state distance is under the same-state threshold τ) against the raw Euclidean baseline already in use.

Cosine/dot-product similarity is *worse* than raw Euclidean, not better; whitening is a wash (0.579 vs. 0.576 precision, well within noise). The metric was never the problem — Euclidean distance in this latent already separates near/far states about as well as any linear transform of it can.

Precision vs. connectivity: the real tradeoff. A corrected calibration formula — $\varepsilon^2 = \frac{\|\mathbf{z}_t - \mathbf{z}_{t+1}\|^2}{d_{\text{eff}} \cdot F_{\chi_{d_{\text{eff}}}^2}^{-1}(q)}$, using the measured adjacent-frame squared displacement directly and the participation ratio d_{eff} in place of d — is always positive by construction (both factors are non-

Table 15: Candidate identification-edge metrics, scored against the ground-truth state oracle (300-episode Push-T landmark set, $n=36,824$).

metric	Spearman(true, metric)	edge precision @ $k=4$
raw Euclidean (current)	0.174	0.576
PCA-whitened	0.204	0.579
per-dim standardized	0.147	—
centered cosine	0.050	—
cosine / dot product	0.037	—

negative), giving $\varepsilon^2=1.32$ at $q=10^{-3}$. Applying it raises identification-edge precision from 57.6% to 96.6%. But the B0 gate metric gets *worse*, not better:

Table 16: B0 gate Spearman under different identification-edge thresholds, same construction otherwise (300-episode landmark set, $k=4$ -capped kNN, identification-edge weight 1.0).

threshold	ε^2	id. edges	unreachable eval pairs	Spearman(true, graph)
$\varepsilon^2 = \infty$ (prior fallback)	∞	72,722	17.3%	0.553
corrected, $q=10^{-3}$	1.32	16,072	99.6%	0.058
adjacent-pair empirical $q_{0.9}$	5.21	32,463	98.0%	0.057
adjacent-pair empirical $q_{0.99}$	14.16	49,297	77.5%	0.191
adjacent-pair empirical $q_{0.999}$	27.20	62,333	40.5%	0.396

Spearman tracks connectivity, not precision: the correctly-calibrated, high-precision threshold shatters the graph into fragments and the gate metric collapses below the no-graph baseline (0.217 raw Euclidean). Precision-weighted edge cost (making a distant edge expensive rather than dropping it, keeping every $k=4$ candidate) was also tested across eight weighting schemes and never beat flat weighting on the already-dense ($\varepsilon^2=\infty$) graph — confirming connectivity, not edge confidence, is the dominant factor once a graph is already connected.

Why the graph fragments. At $q=10^{-3}$ the corrected threshold produces 91 connected components from 300 episodes: 34 episodes end up fully isolated (no cross-episode identification edge at all), 83 episodes merge into one giant component, and the remaining 183 scatter across 56 smaller clusters. This is not primarily a data-sparsity problem — 68.5% of all landmarks genuinely have a true same-state twin somewhere in the other 299 episodes — it is a *recall* problem specific to what this latent encodes. Of landmarks with a genuine true-state twin, the strict threshold finds only 38.7%; widening the kNN search from $k=4$ to $k=1024$ recovers none of the rest (edge count and Spearman both saturate by $k\approx 64$), meaning the missed matches are not merely ranked outside the top few neighbors, they are placed genuinely far away in latent space. The missing 61.3% correlates cleanly with a variable the oracle never sees: agent/block *velocity* (state columns excluded from the position-and-angle oracle). Splitting by velocity-gap quartile between a landmark and its true partner: $P(\text{match found}) = 64.9\%, 47.0\%, 27.0\%, 15.9\%$ (Spearman = -0.40 , $p \approx 0$). LeWM’s latent is trained on a next-frame predictive objective, so two frames sharing position and angle but differing in momentum are, correctly from the encoder’s own perspective, headed toward different futures — the graph is answering a stricter, more predictively-faithful question (“same state, including momentum”) than the one the downstream task actually asks (“same useful position”; the paper’s own live-rollout protocol and this oracle both define goals by position and angle alone, never velocity).

Does a fragmented graph degrade the RL mechanism to imitation? No, for a structural

reason, not a lucky empirical one. HER goals are always drawn from the *same* episode’s own future (`build_her_tuples`); the base $-1/0$ TD objective never reads the graph, and the auxiliary regression target queries $\Phi(s, g)$ only on those same-episode pairs, which a chain of directed transition edges always connects regardless of whether any identification edge exists at all — cross-episode fragmentation is structurally invisible to what actually drives a training step. The real risk is narrower: for an episode with no useful cross-episode shortcut, $\Phi(s, g)$ degrades toward that episode’s own chronological step count, close to redundant with what the $-1/0$ reward already encodes, so the auxiliary loss’s marginal contribution shrinks toward the baseline rather than the mechanism collapsing outright.

Comparing the stitching benefit to Two-Room’s. Restricting Two-Room’s own measured same-episode Spearman gain from identification edges (Section 3.2) to a like-for-like comparison: Push-T’s gain is real but structurally smaller, and for a specific, checkable reason — Two-Room’s graph is always one connected component drawing shortcuts from all 300 episodes, while even Push-T’s best-connected episodes only draw from whichever cluster (2 to 83 episodes) they happen to land in.

Table 17: Same-episode Spearman gain from identification edges, Push-T by component membership. Two-Room, for comparison, is always a single connected component and measures $+0.046$ to $+0.143$ across its 6 tiers (Section 3.2).

group	Push-T gain
isolated episodes (34 of 300)	$+0.015$
giant component (83 episodes)	$+0.036$
small merged clusters (183 episodes across 56 components)	$+0.066$

Isolated and giant-component episodes both fall below Two-Room’s weakest tier; the small merged clusters land inside Two-Room’s range, in some cases above its median tier. The mechanism is the same one validated on Two-Room — real cross-episode shortcut-stitching, not merely connectivity-patching — it is just capped by a smaller shortcut pool on Push-T, on top of (not instead of) the velocity-driven precision/recall tradeoff above.

Takeaways.

- The calibration failure has a specific, checkable cause (a normalization assumption violated by $\approx 0.6\%$ that only matters because $1 - \hat{\rho}$ is itself tiny on Push-T), not a vague “it doesn’t transfer.”
- No alternative distance metric (dot product, cosine, whitening) closes the gap — ruled out directly, not assumed.
- Precision and connectivity trade off sharply on this environment specifically: the statistically correct threshold is a *worse* graph by the gate’s own metric, because it shatters connectivity faster than it improves confidence.
- The recall gap driving fragmentation is attributable, in large part, to velocity — a variable this predictive latent encodes but the position-only oracle does not ask about. Whether to fix the oracle (add velocity) or accept this as inherent to a predictive latent is an open design choice, not yet resolved.
- The RL-training mechanism (Section 2) is structurally immune to cross-episode fragmentation by construction (HER is always same-episode); what degrades under fragmentation is the auxiliary signal’s marginal value on affected episodes, not training correctness.

4.4 A corrected calibration formula, and a B0/B1/B3-style re-run

The fix. `compute_calibration()` (`scripts/common/graph_lib.py`) now computes ε^2 directly from the measured adjacent-pair squared displacement and the covariance’s participation ratio, replacing the $\hat{\rho}/d$ -based formula (Section 4.3). This is always positive by construction, so the $\varepsilon^2=\infty$ fallback — which was never really an approximation of a thresholded search, since it accepts a node’s k nearest neighbors however far away they are — is removed entirely rather than special-cased away. `build_id_edges_faiss_capped`’s kNN search is unchanged: it was already correct (every edge it returns also satisfies the threshold, so its output was always a subset of the exact brute-force scan’s), the fallback was the only path that broke that guarantee. The fix is scoped to `compute_calibration()` specifically, which only the “capped” construction path (currently Push-T only) calls; Two-Room’s “bruteforce” path (`calibrate()`) is left untouched to preserve its validated bit-exact historical numbers — confirmed unchanged ($\varepsilon^2=114.10$) after the change. One narrower side effect: Two-Room’s B4-extension $k\{K\}$ distance-source ablation arms also call `compute_calibration()`, so ε^2 shifts there too ($114.10 \rightarrow 94.06$); the main graph condition and headline B4/B4-extension results (Section 3.6) are unaffected.

B0, re-run. Confirms the diagnosis on the live production path rather than a scratch script: $\hat{\rho}=1.0009$, $\varepsilon^2=1.3226$, 13,750 identification edges, and the gate’s own evaluation collapses to 16 of 4,000 sampled pairs finite (`outputs/ b0_pusht/b0_results.json`). The fix delivers a principled, always-well-defined, subset-preserving construction exactly as specified — it does not, by itself, resolve the connectivity problem diagnosed above.

B1, written and run for Push-T. No unified Push-T B1 existed before this; adapted from Two-Room’s script onto the corrected `common.graph_lib` layer, testing the k -capped construction actually deployed (Two-Room’s B1 tested exact brute force, which is the thing B1 itself found infeasible at scale). Four findings:

- **Edge count scales as $n^{1.61}$** (50/300/1000-episode landmark sets), far better than Two-Room’s uncapped $n^{1.96}$ but not linear either — average degree itself grows with scale ($0.26 \rightarrow 0.75 \rightarrow 1.56$), since a larger dataset increases the odds any given landmark has a genuine revisit.
- **The successor-consistency filter (Section 3.2), which did not help on Two-Room, looks promising here:** of 13,751 identification edges, 82.5% were testable and 86.1% passed; dropping the 1,583 that failed took the (still small-sample) gate Spearman from 0.897 to 0.975. Not yet validated at a sample size large enough to trust the magnitude.
- **Fragmentation is graded across the full q range, not a cliff at $q=10^{-3}$:** even the loosest $q=0.5$ tested gives only 44.7% largest-component coverage and 24.4% cross-episode reachability; both fall monotonically to single digits by $q=10^{-9}$.
- **No pathological hub nodes** (max degree 10, structurally bounded near $2k$) — the k -cap does what it is meant to.

B3, re-run. The prior 30-run sweep (5 tiers $\times$ 2 conditions $\times$ 3 seeds, Section 4.2) was built entirely under the removed $\varepsilon^2=\infty$ fallback; those checkpoints and the stale per-tier Φ cache were preserved (`outputs/b3_pusht/{checkpoints,tier_cache}_pre_calib_fix/`) rather than overwritten, and the sweep re-launched from scratch under the corrected, fragmented-but-principled graph, same tiers/seeds/step budget.

Baseline is essentially unchanged tier-for-tier (≤ 0.001 everywhere) — exactly what the mechanism argument above predicts, since baseline never reads the graph and the two sweeps share seeds;

Table 18: B3 re-run under the corrected calibration: peak held-out-test Spearman, mean over 3 seeds. Baseline never reads the graph, so it serves as a same-seed control between the two sweeps.

tier	baseline (prior $\rightarrow$ re-run)	auxphi (prior $\rightarrow$ re-run)	Δ (prior)	Δ (re-run)
expert_10	0.148 $\rightarrow$ 0.1476	0.178 $\rightarrow$ 0.1772	+0.030	+0.030
expert_25	0.284 $\rightarrow$ 0.2838	0.330 $\rightarrow$ 0.3255	+0.046	+0.042
expert_50	0.128 $\rightarrow$ 0.1282	0.182 $\rightarrow$ 0.1745	+0.054	+0.046
expert_100	0.074 $\rightarrow$ 0.0738	0.144 $\rightarrow$ 0.1293	+0.070	+0.056
mixed	0.391 $\rightarrow$ 0.3909	0.418 $\rightarrow$ 0.4638	+0.027	+0.073

this is a useful same-seed control confirming nothing else about the setup moved between runs. Auxphi still beats baseline at all 5 of 5 tiers under the far more fragmented graph, but the per-tier margin is not uniformly larger or smaller: it shrinks at expert_25/50/100 (most at expert_100, +0.070 $\rightarrow$ +0.056) and grows sharply at mixed (+0.027 $\rightarrow$ +0.073, now easily the largest margin of any tier in either sweep). Connectivity collapsing from 82.7% reachable (Section 4.3’s $\varepsilon^2=\infty$ row) to 0.4% reachable did not break the auxiliary signal, consistent with the structural argument that same-episode HER pairs never depend on cross-episode connectivity — but it did not leave tier-by-tier magnitudes untouched either, and the shift has no settled explanation yet: mixed’s combination of real and noisy episodes may interact with the corrected graph’s episode-loop-closure edges (Section 4.3) differently than expert-only tiers do, but this is a hypothesis, not a confirmed mechanism.

4.5 The live-rollout evaluation protocol

Motivation. Every Push-T actor described so far was evaluated on arbitrary cross-episode (s, g) pairs from the B0/B1 Spearman eval set, with a 250-step budget (`eval_protocol=cross_episode`) — the original convention, ported unchanged from Two-Room. Every condition scored 0.000 under it (Section 4.2). Before reading that as “the method fails on Push-T,” the protocol itself needed checking against how the LeWM paper actually measures its own Push-T policies (Fig. 6: GCBC 75%, GCIVL 33%, GCIQL 20%, Random 2%): `stable-worldmodel`’s `scripts/plan/eval_ff.py` (App. F.1) starts from a random dataset state, sets the goal to the state exactly 25 timesteps later in the *same* trajectory, and allows 50 env steps to reach it — success is the environment’s own `terminated` flag.

Algorithm 14 `eval_protocol=same_episode` (`config/graph/actor.yaml`, now default)

- 1: start $\leftarrow$ a row sampled from `eval_pool` (`dataset` = the full h5 minus the tier’s train episodes, or `test_episodes` = the tier’s own held-out episodes)
 - 2: goal $\leftarrow$ the state `env.paper_goal_offset`= 25 steps later in the same episode
 - 3: run the actor for up to `env.paper_eval_budget`= 50 steps; success = env’s own `eval_state()`
-

Results. All runs below use the exact same actor-training loop and hyperparameters as Two-Room’s B3/B4 (`aux_lambda`=0.3, `her_goal_gamma`=0.9, `aux_standardize`=false, `awr_normalize_adv`=false, `aux_cross_lambda`=0.0 — i.e. “vanilla” auxphi, none of the Push-T-specific knobs introduced in Section 4.6 below), trained via `scripts/actor_train.py env=pusht tier=<tier> variant=<baseline|auxphi> seed=0` and scored via `scripts/actor_rollout_eval.py` with the config shown above, `eval_seed`=42, $n = 100$ episodes (random baseline: $n = 20$,

policy=random, no checkpoint).

Table 19: Live-rollout success rate under the paper’s own protocol (eval_protocol=same_episode, eval_pool=dataset, eval_seed=42).

tier	condition	select	n	success rate
expert_25 (20 train eps)	random policy	—	20	15%
expert_25	baseline	rho	100	7%
expert_25	auxphi	rho	100	7%
expert_25	baseline	success	100	7%
expert_25	auxphi	success	100	7%
expert_25	baseline, $\gamma_{\text{HER}} = 0.96$	rho	100	2%
expert_25	baseline, $\gamma_{\text{HER}} = 0.96$	success	100	5%
expert_100 (80 train eps)	baseline	rho	100	22%
expert_100	auxphi	rho	100	26%

Takeaways.

- The 0.000 result was a protocol artifact, not a method failure: the same expert_100 checkpoints that scored 0 under **cross_episode** score 22–26% under the paper’s own protocol, comfortably above the measured random floor (15% on this pool; the next subsection reports a lower 4–9% floor on other pools — the floor itself is pool-dependent and must be measured alongside any headline number, not assumed).
- At expert_25, baseline and auxphi are statistically indistinguishable (both 7%, both selection rules) — essentially tied at the random floor with only 20 train episodes. Widening the HER goal horizon ($\gamma_{\text{HER}} : 0.9 \rightarrow 0.96$) did not help; it made the baseline worse (7% $\rightarrow$ 2–5%), so the goal-sampling distribution is not the bottleneck at this data scale.
- At expert_100, auxphi’s edge over baseline (26% vs. 22%) reappears once there is enough data to separate the two conditions from noise — consistent with, though smaller in magnitude than, the Spearman-proxy gap measured for the same tier (Section 4.2: +0.070).
- Data scale, not any of the algorithmic knobs tried so far, is the dominant lever toward the paper’s own 20–75% range — see the next subsection.

4.6 Diagnosing the apparent underperformance

Two plausible explanations, tested directly and refuted. Diagnostics in `scripts/investigations/pusht_100` run against the `mixed_large` tier (50 real expert + 100 `WeakPolicy`-rollout episodes, 78% of frames from the wandering, task-*never*-succeeding policy).

- “*The privileged-state oracle is bogus for a contact-manipulation task.*” Graded against real expert step-gaps (a genuine dynamical ground truth on near-optimal data) rather than assumed correct: Spearman = +0.90. It is fine.
- “*The latent graph itself is bad on Push-T.*” It is not — Spearman = +**0.9952** against expert step-gaps, better than Two-Room’s own headline number (Section 3.1). Median `phi_dist` matches the true step-gap almost exactly across scales: gaps 1/2/5/10/25/50/100 $\rightarrow$ `phi_dist` 1.0/2.0/5.0/10.0/25.0/50.0/99.0.

- “AWR’s importance weights saturate into uniform behaviour cloning.” Only 4.4% of weights hit the clip (`awr_weight_clip=100`) — not the cause either.

The real cause. `pseudo_v` (the auxiliary regression target) is derived from graph geodesic distance, which measures *how many steps the data actually took* — a behavioural distance. IQL’s expectile-0.9 value function is trained to learn the *optimal* cost-to-go. The two coincide on near-optimal data (exactly why the mechanism works cleanly on Two-Room and on Push-T’s own expert-only tiers) and diverge sharply once the data contains wandering, never-succeeding trajectories. Measured directly on `mixed_large`’s own HER pairs: median `pseudo_v` = -5.85 vs. the baseline’s TD-learned V median = -2.76 ; `auxphi`’s own V gets dragged to -4.98 . The downstream damage is concrete: the AWR advantage picks up a systematic negative bias (mean -0.24 for `auxphi` vs. -0.69 for baseline), which *inverts the actor’s data preference* — the ratio of mean importance weight on expert vs. noisy transitions is 1.03 for baseline (neutral) but only **0.35** for `auxphi`, i.e. `auxphi` weights the never-succeeding `WeakPolicy` data $\approx 3\times$ more than expert data. This is also, run in reverse, exactly why the mechanism works on Two-Room: its “noisy” tier is a noised *expert* policy that still solves the task, so behavioural and optimal cost-to-go never meaningfully diverge there.

Data composition is the dominant lever — more than any algorithmic knob tried.

Table 20: Baseline live-rollout success rate by data composition (paper protocol, `eval_protocol=same_episode`). Paper’s own Push-T numbers, same protocol family: GCBC 75%, GCIVL 33%, GCIQL 20%, Random 2% (measured 4–9% here on a comparable pool).

tier	composition	baseline success
<code>mixed_large</code>	50 expert + 100 <code>WeakPolicy</code> eps (78% wandering)	26%
<code>expert_1000</code>	1000 pure expert eps	54%

A pure-expert-data baseline (54%) already beats both of the paper’s offline-RL baselines (GCIQL 20%, GCIVL 33%) under their own protocol — “our method underperforms on Push-T” is not a supportable claim once tier composition is held fixed and stated.

Two config-gated fixes, both defaulting to false so no previously-published Two-Room number moves (`config/graph/actor.yaml`): `awr_normalize_adv` (z -score the AWR advantage before `exp()`, so a biased/drifted V cannot silently rebias every actor weight) and `aux_standardize` (regress V onto the z -scored `pseudo_v` instead of its raw value, learning the graph’s *ordering* without importing its behavioural absolute scale).

Table 21: `mixed_large` `auxphi` live-rollout success under each fix in isolation (`env=pusht` `tier=mixed_large` `variant=auxphi`; baseline reference 26%, from above).

condition	success rate
<code>auxphi</code> (vanilla, both knobs false)	14%
<code>auxphi</code> + <code>aux_standardize=true</code>	16%
<code>auxphi</code> + <code>awr_normalize_adv=true</code>	21%

Takeaways.

- Never quote a Push-T policy number without stating tier composition — expert-only vs. mixed changes baseline success by $2\times$, more than any method difference measured here.

- Auxphi’s benefit is conditional on the data being near-optimal; on a new environment it should be tested on a pure-expert tier first, since a mixed/degraded tier can make it look like an outright regression when the real cause is a distance-scale mismatch, not a broken mechanism.
- The graph itself is the best-validated component in this whole pipeline on Push-T (+0.9952 vs. true step-gaps) — it was never the suspect.
- `awr_normalize_adv` recovers more of the gap than `aux_standardize` (21% vs. 16%, vanilla 14%), suggesting the AWR-advantage bias is the larger of the two mechanisms by which the behavioural/optimal mismatch does damage, though neither fix fully closes it back to a clear win over the 26% baseline.

4.7 Within-episode identification edges

Change. `build_id_edges_faiss_capped` (the default, “capped” identification-edge construction used by Push-T) used to discard *every* same-episode k NN neighbour outright, regardless of how far apart in time the two frames were. Fixed to accept a `min_step_gap` parameter (default 5): a same-episode neighbour is now kept as an identification edge as long as its step gap is $\geq \text{min_step_gap}$ — a small exclusion window avoids duplicating transition edges ($\text{gap} = 1$) or near-duplicate consecutive frames, while letting a wandering trajectory that loops back near an earlier state get the shortcut its own revisit actually earns. Two-Room’s `bruteforce` identification-edge path is untouched, to protect its bit-exact historical numbers (Section 2).

Result (gate check, Push-T `mixed` tier: 50 expert + 50 `WeakPolicy` episodes, 19,017 landmarks, 300k random eval pairs, oracle = z -scored Euclidean over projected state):

Table 22: Effect of allowing same-episode identification edges (`min_step_gap=5`).

construction	id. edges	same-episode Spearman ($n=3379$)	cross-episode Spearman (n)
OLD (same-episode excluded)	3,227 (0 same-ep)	0.560	−0.178
NEW (<code>min_step_gap=5</code>)	13,835 (10,608 same-ep)	0.678	−0.168

Confirms the intended, targeted effect: +10,608 legitimate same-episode loop-back shortcuts and +0.12 Spearman specifically on same-episode pairs, where the fix should matter. Cross-episode/overall correlation is negative both before and after — a separate, already-diagnosed issue (Sections 4.3 and 4.6), not something this fix targets.

Live-rollout check. Full actor training with the fix in place (`scripts/actor_train.py env=pusht tier=mixed variant=auxphi seed=0 eval_protocol=cross_episode, 20k steps, local GPU`): peak held-out value-Spearman 0.375 at step 200. `actor_rollout_eval.py ... select=rho n_episodes=100` under the same `cross_episode` protocol (fixed held-out (s, g) pairs, 250-step budget): **0/100** — matching the pre-existing finding (Section 4.2) that every Push-T actor scores 0 under this protocol.

Takeaways.

- A real, targeted improvement to identification-edge quality on wandering data, verified at the graph-construction level (+0.12 same-episode Spearman) — but not a fix for the separate `cross_episode`-protocol 0% finding: arbitrary cross-trajectory goals for a contact-manipulation task, at a 250-step budget, are just hard regardless of graph quality.

- This fix is now baked into the default Push-T construction path (no config flag) — every Push-T result reported after this point, including Section 4.9’s checkpoints, uses it.

4.8 Directed transition edges and a cross-episode auxiliary term

Two further changes to `scripts/common/graph_lib.py` / `actor_train.py`, tested on Push-T’s `mixed` tier.

1. Directed transition edges. `build_weighted_graph` used to add every transition edge in both directions with equal weight. An env step is not generally reversible (pushing a block cannot be undone by reversing the action), so an undirected geodesic overstated closeness in the direction that matters. Transition edges are now one-directional (earlier-step $\rightarrow$ later-step only); identification edges stay symmetric. Every `dijkstra(...)` call across the active pipeline was switched to `directed=True` accordingly. **This also changes Two-Room’s historical bit-exact B0–B4 numbers** (Section 3.1–Section 3.6 no longer reproduce to full precision) — accepted deliberately, since Two-Room’s transitions are only “roughly” reversible and preserving exact reproduction was never a design goal once a genuinely more accurate graph is available. The six historical re-export shims and `scripts/investigations/*` still call `dijkstra(..., directed=False)` and so silently keep the old symmetric semantics — correct, since those scripts are explicitly out of scope for edits.

2. A cross-episode auxiliary term. The existing auxiliary-regression target (Algorithm 2) only ever reads Φ at HER pairs, which are same-episode by construction — so V never saw the graph’s cross-episode stitching during training, even though that is the graph’s entire reason for existing over raw Euclidean distance. Fix: a second, independently-sampled aux term at pairs drawn from the graph’s own identification edges that cross an episode boundary (real stitching links the graph itself found, not arbitrary far-apart pairs, so Φ stays small by construction — 3,860 valid cross-episode training pairs on the `mixed` tier, mean $\Phi = 1.00$, i.e. mostly single direct edges). Weighted by a new `aux_cross_lambda` (default 0.0 = off, exact historical behaviour).

Table 23: `mixed` tier, seed 0, `eval_protocol=cross_episode`, live-rollout $n = 100$ (Ada gnode079).

condition	peak value-Spearman	live-rollout success
historical (undirected graph, no within-ep id. edges, no cross aux)	0.4219	0/100
directed graph + within-ep id. edges, <code>aux_cross_lambda=0</code>	0.3764@250	0/100
+ <code>aux_cross_lambda=0.3</code>	0.3544@200	0/100

Takeaways.

- Neither change flips the `cross_episode`-protocol result in this single-seed test. This is consistent with, not contrary to, Section 4.7: every known Push-T condition is floored at 0% under this specific protocol, so it cannot show an improvement even if the underlying mechanism works.
- The peak-Spearman drop (0.42 $\rightarrow$ 0.38 $\rightarrow$ 0.35) is not apples-to-apples and should not be read as these changes hurting — directed transitions change what Φ measures (a harder, more accurate directional distance), not a comparable-scale regression.
- The more informative next test is `eval_protocol=same_episode` (where baseline already scores above floor, Sections 4.5 and 4.6) with both fixes in place — the checkpoints used in Section 4.9

below were trained after both landed (they are now unconditional defaults, not opt-in flags, except `aux_cross_lambda`, which stays off there too).

4.9 A stitching-specific showcase, with safeguards against p-hacking

Motivation. An arbitrary cross-episode (s, g) pair is usually just *far* in a 250-step-budget sense, regardless of whether the graph could find a shortcut — so a plain cross-episode eval is dominated by raw long-horizon difficulty, not by whether the graph can stitch. A first naive attempt confirmed this: 100 arbitrary cross-episode pairs, identical for both actors, scored baseline 8% / auxphi 9% — both near the random floor, telling us nothing about stitching specifically. The actual claim under test — that auxphi’s edge over baseline should be concentrated on cross-episode pairs the graph itself says are close via identification edges (true stitching opportunities), and should vanish on pairs it says are far (a negative control) — needs pairs stratified by graph distance, not sampled arbitrarily.

Methodology, with five safeguards fixed before looking at any outcome (to guard against post-hoc bucket-tuning), implemented in `scripts/investigations/pusht_lowscore/stitching_eval.py`:

1. Bucket boundaries fixed in advance and never tuned: near < 20 , mid $[20, 50)$, far ≥ 50 , in true graph-distance (hop-count) units.
2. Every bucket reported, including far as the negative control.
3. Success is the real physical `eval_state()` criterion, never Φ itself — Φ only decides which pairs to ask about.
4. The near bucket is independently cross-checked against the *true* oracle distance (`mech.build_true_distance_or` privileged z -scored state distance, computed with no reference to the graph) — rules out “the graph invented a shortcut that isn’t real.”
5. The near bucket is also reported by hop count (since every edge has weight 1, Φ is the shortest-path hop count) and raw latent Euclidean distance — separates genuine multi-hop stitching from a trivial near-duplicate pair findable by plain nearest-neighbour lookup.

Run as `python scripts/investigations/pusht_lowscore/stitching_eval.py env=pusht tier=<expert_1000|mixed_large>`: 6,000 cross-episode candidate pairs sampled uniformly at random (seed 12345) from the tier’s own landmark set, bucketed by bounded-Dijkstra Φ (`limit=100`, `phi_mode=sparse`); 40 identical pairs per bucket run through *both* the baseline and auxphi checkpoints for that tier (same seed, same env, budget = 250 steps = `mech.max_episode_steps`, the full budget rather than the paper’s 50-step same-episode budget, since these are deliberately far cross-episode goals). Checkpoints are the vanilla-auxphi ones trained after the fixes in Sections 4.7 and 4.8 landed as defaults (`aux_cross_lambda=0`, `aux_standardize=false`, `awr_normalize_adv=false`), selected by `select=rho`.

Results.

Both tiers’ near buckets pass: genuinely multi-hop (median 14–15 real graph hops, not a single identification-edge shortcut), oracle-close (small median true state distance, positively correlated with Φ), and non-trivial (0% within 2 hops of a plain nearest-neighbour match).

This is a genuine negative result for the stitching-showcase hypothesis, reported as measured rather than reframed: on `expert_1000`, where both actors clear the floor at all (near-bucket baseline 45%),

Table 24: Safeguards 4/5: near-bucket validation against the independent true oracle and hop-count/Euclidean controls.

tier	near-bucket n	oracle_dist median	Spearman(Φ , oracle)	hop-count median	Euclidean median	% trivial ($\Phi \leq 2$)
expert_1000	50/6000	0.823	+0.4685	14.0	16.29	0.0%
mixed_large	499/6000	2.651	+0.2833	15.0	12.34	0.0%

Table 25: Live-rollout success by bucket, identical pairs for both variants, 40/bucket, 250-step budget.

tier	bucket	Φ range	baseline	auxphi	Δ
expert_1000	near	[0, 20)	45.0%	37.5%	-7.5%
expert_1000	mid	[20, 50)	7.5%	7.5%	+0.0%
expert_1000	far	[50, ∞)	2.5%	2.5%	+0.0%
mixed_large	near	[0, 20)	0.0%	0.0%	+0.0%
mixed_large	mid	[20, 50)	0.0%	0.0%	+0.0%
mixed_large	far	[50, ∞)	0.0%	0.0%	+0.0%

baseline outperforms auxphi by 7.5 points on exactly the pairs the graph says are the best stitching opportunities — the opposite of the hypothesis. Mid and far buckets are statistically tied and near the random floor for both variants, as expected for a negative control. On **mixed_large**, every bucket floors at 0% for both variants, so the eval has no discriminating power there at all (consistent with Section 4.6’s finding that **mixed_large**’s wandering-data composition already suppresses auxphi’s advantage broadly, not just on stitching pairs specifically).

Takeaways.

- The near buckets are methodologically sound (safeguards 4/5 pass on both tiers), so the null/negative result is not an artifact of accidentally picking trivial or mislabelled pairs.
- The result is consistent with Section 4.6’s mechanism: the auxiliary regression target is a *behavioural* distance (how many steps the data took along a path the graph found), not the actor’s own optimal cost-to-go. A genuine cross-episode stitching pair is exactly where this mismatch should matter most, since the data itself never took that shortcut — so it is a plausible, not yet independently confirmed, explanation for why baseline (which never queries the graph at cross-episode pairs at all) does no worse, and here slightly better, than auxphi on the near bucket.
- The as-yet-untried **aux_cross_lambda** > 0 (Section 4.8) is the one mechanism in the current pipeline specifically designed to expose V to the graph’s cross-episode opinion during training rather than only same-episode HER pairs — a direct, motivated next experiment for this eval, not yet run.
- **expert_1000**’s baseline being the strongest performer in the whole table (45% near-bucket) is itself informative: with enough pure-expert data, plain goal-conditioned BC/TD needs no graph at all to find real cross-episode shortcuts through the data manifold — the mechanism, if it helps at all, would need to earn its keep against a substantially stronger, not weaker, baseline than the small-data tiers used elsewhere in this document.

4.10 A looser, empirically-set identification threshold, swept across data scale

Why a 99th-percentile threshold. Section 4.3 showed that ε^2 (however it is set) is trying to separate two populations whose latent distances barely overlap: real same-episode 1-step transitions cluster near $\varepsilon^2 \approx 1$ –2, unrelated cross-episode pairs cluster near $\varepsilon^2 \approx 386$, four orders of magnitude apart. Directly plotting both distributions confirms the calibrated threshold ($\varepsilon^2 \approx 1.3$) sits at only the 50th percentile of real transitions — it rejects half of all genuine 1-step moves as “not similar enough,” which is a far stricter bar than “is this a plausible identification.” A threshold set at the *empirical* 99th percentile of real transition distances is a direct, model-free way to ask “how loose can the threshold get while still keeping it inside the transition population”: measured on the 300-episode reference set, this is $\varepsilon^2 = 14.16$ — notably not the same number `compute_calibration`’s own chi2-formula gives at $q = 0.99$ (3.12; the real distribution has heavier tails than the chi2 approximation assumes, a $4.5\times$ gap). At $\varepsilon^2 = 14.16$, only 0.016% of random cross-episode pairs would be accepted, so this is still a highly selective threshold, just $\sim 10\times$ looser than the calibrated default.

Algorithm 15 Methodology: retraining setup and the 99th-percentile threshold experiment

- 1: **Retraining changes** (all tiers/variants below): $\gamma_{\text{HER}} : 0.9 \rightarrow 0.99$ (mean HER goal offset $10 \rightarrow 100$ steps, matching the paper’s own GCIQL critic value); $n_{\text{steps}} : 20000 \rightarrow 35000$; rollout-check episodes $12 \rightarrow 20$; success-checkpoint tie-breaking changed from strict $>$ to $\geq$ (prefers the later, more-trained checkpoint on a tie); only `select=success` evaluated going forward (`select=rho` tracks a metric already shown, Section 4.3, to be dominated by cross-episode pairs the value function never trains on)
 - 2: **Identification threshold:** `id_eps2_override=14.16` (new `common.graph_lib` parameter, bypasses `compute_calibration` entirely for a fixed, empirically-set ε^2 – Section 4.10 above)
 - 3: **Tiers:** `expert_{100,300,1000}`, both variants, 3 seeds
 - 4: **Eval:** live-rollout success rate, 100 episodes, at three horizons – `goal= {25,50,100}` steps with a matching $2\times$ budget (`{50,100,200}` steps)
-

Results.

Table 26: Live-rollout success rate at $\varepsilon^2 = 14.16$ (empirical 99th-percentile threshold), mean over 3 seeds except where noted.

tier / goal	baseline	auxphi	winner
<code>expert_100 / 25</code>	$21.7\% \pm 2.1$	$18.3\% \pm 2.1$	baseline
<code>expert_100 / 50</code>	$11.0\% \pm 1.4$	$5.7\% \pm 2.1$	baseline
<code>expert_100 / 100</code>	$2.0\% \pm 1.4$	$0.7\% \pm 0.9$	baseline
<code>expert_300 / 25</code>	$31.7\% \pm 1.7$	$33.3\% \pm 5.2$	tie
<code>expert_300 / 50</code>	$13.3\% \pm 2.6$	$12.7\% \pm 3.4$	tie
<code>expert_300 / 100</code>	$8.3\% \pm 1.9$	$5.7\% \pm 1.7$	baseline
<code>expert_1000 / 25</code>	$35.3\% \pm 4.6$	$43.7\% \pm 2.5$	auxphi
<code>expert_1000 / 50</code>	$19.7\% \pm 4.2$	$22.7\% \pm 1.7$	auxphi
<code>expert_1000 / 100</code>	$6.7\% \pm 2.6$	$9.0\% \pm 0.8$	auxphi

All rows above are the complete $n=3$ result. `expert_300`’s baseline row initially stalled at seed 2 while building the dense $N \times N$ phi-distance matrix and had to be re-launched — root cause below.

A clean scale trend. Auxphi loses at expert_100 (all 3 horizons), is statistically tied at expert_300, and wins outright at expert_1000 (all 3 horizons, by 6–8 points absolute). This is the same explanation already proposed on other grounds (Section 4.3’s same-episode stitching-benefit comparison): more episodes means more chances for a real cross-episode revisit to exist at all, even at a fixed threshold – so the auxiliary signal has more genuine substance to work with as data scales, independent of anything about the threshold value itself. This sweep is the first place that trend shows up directly on live-rollout success rather than a same-episode Spearman proxy.

A real inefficiency, caught and fixed. The stalled expert_300/baseline/seed-2 run traced to actor_train.py’s tier setup (get_setup/build_tier_setups) always building the dense phi-distance matrix by default, regardless of which variant will use it – baseline sets aux_lambda=0 and never reads phi_dist at all, but nothing skipped the (expensive, $O(n^2)$ -memory) construction for it. With three combos per tier building the same multi-GB matrix concurrently, the baseline runs were pure waste that happened to tip a shared-memory budget over the edge. The existing need_graph=false config flag (already used for very large expert-only tiers, Section 2) fixes this when passed explicitly for baseline runs; it was not passed in this sweep’s launch script and is now corrected there for future runs.

The same sweep at the default calibrated threshold. To check whether the scale trend above is specific to the looser $\varepsilon^2 = 14.16$ threshold, the identical sweep (same tiers, retraining changes, and eval horizons) was re-run at compute_calibration’s own default threshold instead ($\varepsilon^2 \approx 1.3$, run_tag=_defaulteps, no override).

Table 27: Live-rollout success rate at the default calibrated threshold ($\varepsilon^2 \approx 1.3$), mean over 3 seeds.

tier / goal	baseline	auxphi	winner
expert_100 / 25	23.7% $\pm$ 3.9	27.0% $\pm$ 1.6	auxphi
expert_100 / 50	9.7% $\pm$ 2.9	7.3% $\pm$ 2.5	baseline
expert_100 / 100	1.7% $\pm$ 0.9	1.3% $\pm$ 0.5	tie
expert_300 / 25	30.7% $\pm$ 1.2	30.3% $\pm$ 2.6	tie
expert_300 / 50	12.3% $\pm$ 1.7	14.0% $\pm$ 2.2	tie
expert_300 / 100	6.3% $\pm$ 0.9	5.7% $\pm$ 1.7	tie
expert_1000 / 25	33.7% $\pm$ 4.2	38.7% $\pm$ 1.7	auxphi
expert_1000 / 50	17.7% $\pm$ 1.9	26.7% $\pm$ 5.4	auxphi
expert_1000 / 100	7.0% $\pm$ 0.8	8.7% $\pm$ 0.9	auxphi

All rows above are the complete $n=3$ result. expert_300/auxphi/seed 1 initially crashed reading a half-written phi-distance cache file (three auxphi seeds for the same tier tried to build and cache that matrix concurrently) and was retrained separately once the cache was cleared.

The scale trend replicates. The same pattern holds at the default threshold: auxphi is mixed-to-behind at expert_100 (wins the shortest horizon, loses the other two), a statistical tie at expert_300, and a clean win at expert_1000 across all three horizons. So the earlier trend was not an artifact of the looser $\varepsilon^2 = 14.16$ threshold – identification-edge density growing with data scale is what drives it, independent of exactly where the threshold is set, consistent with Section 4.3’s explanation that more episodes means more genuine cross-episode revisits for the auxiliary signal to learn from.

4.11 Testing the advantage-drift hypothesis: `aux_standardize` and `awr_normalize_adv`

Why this test. A comment already in `actor.yaml`, from an earlier investigation, records a measured advantage drift on Push-T: mean AWR advantage -0.689 (baseline) vs. -0.236 (auxphi) on `mixed_large`. Since AWR’s policy weight is $\exp(\beta \cdot \text{advantage})$ clamped at `awr_weight_clip`, a uniform shift in advantage is not cosmetic: it changes how many actions saturate at the clamp, which flattens exactly the fine-grained preference ranking AWR depends on. Two existing (never-enabled) config flags target this directly: `aux_standardize` regresses V onto the graph pseudo-value’s z-scored *ordering* rather than its raw behavioral-distance scale; `awr_normalize_adv` z-scores the advantage itself immediately before the $\exp(\cdot)$ weighting, so a constant drift from the aux term cannot silently change the clamp population. Neither was enabled in any Push-T run before this one, including Section 4.10’s two sweeps.

Algorithm 16 Methodology: identical to Section 4.10’s default-threshold sweep, plus the two flags

- 1: **Unchanged from `run_defaulteps_sweep.sh`:** $\epsilon^2 \approx 1.3$ (`compute_calibration` default, no override); $\gamma_{\text{HER}} = 0.99$; $n_{\text{steps}} = 35000$; rollout-check episodes = 20; tiers `expert_{100,300,1000}`, both variants, 3 seeds; eval at goal= {25, 50, 100} with matching $2 \times$ budget; `select=success` only
 - 2: **Added:** `aux_standardize=true` `awr_normalize_adv=true`
 - 3: **Distinct `run_tag=_defaulteps_stdadv`** so this cannot collide with or resume from the plain `_defaulteps` checkpoints; same tier-cache directory reused (the graph/ Φ matrix is unchanged, only the training loss is)
-

Results.

Table 28: Live-rollout success rate at the default calibrated threshold, with `aux_standardize/awr_normalize_adv` enabled, mean over 3 seeds.

tier / goal	baseline	auxphi	winner
expert_100 / 25	26.7% ± 0.5	22.7% ± 1.9	baseline
expert_100 / 50	10.7% ± 2.1	10.0% ± 0.8	tie
expert_100 / 100	2.0% ± 0.8	1.3% ± 1.2	tie
expert_300 / 25	38.3% ± 3.3	34.0% ± 6.5	tie
expert_300 / 50	18.3% ± 2.4	13.3% ± 3.3	baseline
expert_300 / 100	4.3% ± 0.9	6.3% ± 1.2	tie
expert_1000 / 25	48.0% ± 3.6	41.0% ± 5.9	baseline
expert_1000 / 50	26.7% ± 3.3	25.3% ± 3.9	tie
expert_1000 / 100	7.7% ± 1.9	10.7% ± 3.7	tie

The hypothesis does not hold up — if anything, it runs backward. Under the plain `_defaulteps` sweep (Section 4.10’s second table), auxphi had 4 clear wins (expert_100/25, and all three expert_1000 horizons by 5–9 points) against 1 clear baseline win and 4 ties. With the drift fix applied, auxphi has *zero* clear wins anywhere: baseline picks up 3 clear wins (expert_100/25, expert_300/50, expert_1000/25) and everything else is now a tie. Most notably, expert_1000’s clean 3-for-3 auxphi sweep — the strongest evidence for the graph signal anywhere in the Push-T results, and the basis for Section 4.10’s scale-trend claim — is gone: baseline now leads or ties on all three of its horizons. Both baseline’s and auxphi’s absolute numbers move under the fix

(advantage normalization is not auxphi-specific, since `awr_normalize_adv` applies to both variants), but baseline moves up more often and by more, particularly at the larger tiers and shortest horizon (expert_1000/25: +14 points for baseline vs. +2 for auxphi). The advantage-drift measured on `mixed_large` was real, but correcting it did not unlock a hidden auxphi advantage — it mostly stabilized baseline’s own AWR extraction, which had nothing to do with the aux term in the first place. This narrows, rather than resolves, the open question from Section 4.3: the scale-trend explanation (more episodes $\rightarrow$ more genuine stitching substance) is not wrong on its own terms, but it was measured under a training setup with an uncontrolled confound, and the one sweep that isolates that confound erases the effect it was meant to explain.

5 Risks

1. **Potential-based shaping (mode 3) fired, and not the way this paragraph predicted — but the underlying idea recovers under a different consumption mode.** The hypothesis here was that shaping would help *most* in degraded (scarce/mixed) data tiers, where stitching matters most and TD has least to work with, with clean-data results serving as a control. The measured result was the reverse: mode 3 won only with 100 clean episodes, lost at every smaller size, and lost *worse* as more heterogeneous data was added (Section 3.4). That was a confirmed risk in the direction least favorable for deployment. What resolved it was not tuning mode 3 (scaling its strength down closes most, not all, of the gap) but switching to mode 2 — using Φ as a direct auxiliary regression target instead of folding it into the TD-bootstrapped reward (Section 2). Mode 2 wins at 5 of 6 tiers, by a margin that grows with data. The remaining risk is narrower: it is now about *which consumption mode* to recommend by default, not about whether the graph-distance signal is worth anything — mode 3’s policy-invariance guarantee is real but does not, by itself, protect against noise compounding through TD.
2. **The graph is dense, and it gets denser faster than the data does.** Measured, not estimated: identification edges scale as landmark-count^{1.96} on Two-Room (Section 3.2), extrapolating to $\sim 1.5 \times 10^{10}$ edges at Push-T’s scale. Plain landmark subsampling delays this but does not fix the scaling law. The construction needs a top- k -per-node cap (bounding degree directly, turning quadratic growth into $O(nk)$) before any Push-T attempt; this is now a blocking prerequisite for the Push-T work (Section 4), confirmed in B1 rather than assumed.
3. **Physically close, dynamically separated.** States on opposite sides of a thin wall can be falsely identified — the measured false-edge rate is 2–7% depending on q . Successor-consistency filtering (if i, j are the same state under similar actions, their recorded successors must also pass the test) was implemented and tested in B1: it removed 8.4% of identification edges but left the false-edge rate and downstream Spearman unchanged, because only 25.6% of edges had similar-enough actions to even test — Two-Room’s target is invisible in the pixel observation, so same-position pairs from different episodes are typically executing unrelated goal-directed policies. The false-edge problem itself is real and unresolved; this particular fix does not solve it here, and should not be assumed to transfer to an environment where the goal is visible.
4. **Novelty is narrower than it looks.** SoRB (Eysenbach et al. 2019) and semi-parametric topological memory (Savinov et al. 2018) both build graphs over experience; both learn their edge predicate from online or reward-labelled data. The specific claim here is narrower and should be stated narrowly: *a reward-free, frozen, pretrained latent metric with a distributionally-calibrated identification threshold, deployed as a policy-invariant potential*. B2 is what decides

whether even that survives.

6 Resources

Two-Room data and the pretrained checkpoint are downloaded and working; B0’s encoded landmarks are cached (`outputs/b0_tworoom/landmarks_ep300.npz`) and reusable without re-encoding. Push-T data is already local. Baseline checkpoints (PLDM, LeJEPa, DINO-WM, GCIQL, GCBC) are published for Two-Room, so B2 and B3 need no baseline training from scratch. Encoding runs at ~ 600 frames/s on the local GPU; B0 end-to-end took under two minutes with landmarks cached.

The loader (`scripts/tworoom_lewm_loader.py`) contains a state-dict key remap needed because the installed `transformers` renamed ViT block internals relative to the published checkpoint; verified as a pure rename before use. Any other pretrained LeWM checkpoint will need the same treatment.